Сравнение политик вытеснения (LRU, LFU, Random, NoEviction) в потокобезопасном in-memory кэше

на различных профилях нагрузки

Панфилов В. А. Васильев Д. А. Величко Д. И. Ладаев Т. А.
Волкогонов Л. К.

24 июня 2026 г.

https://github.com/vlklk/team19-key-value-cache

План

1. Цель
2. Теория
3. Архитектура
4. Методология
5. Результаты
6. Выводы

Цель и задачи

Цель. Определить оптимальную политику вытеснения для каждого типа нагрузки по объективным метрикам (hit ratio, throughput, latency).

Задачи:

- потокобезопасный in-memory кэш на C++23;
- четыре политики вытеснения: LRU, LFU, Random, NoEviction;
- механизм TTL;
- параметризуемый бенчмарк (6 профилей нагрузки);
- эксперименты и анализ результатов.

Главный вывод. Универсально лучшей политики нет — выбор зависит от профиля нагрузки.

Политики вытеснения

Эталон — алгоритм Беладі (вытеснять то, что понадобится позже всех); на практике недостижим, служит верхней оценкой hit ratio.

- **LRU** — по давности обращения. Слабость: ломается на сканировании длиннее кэша.
- **LFU** — по частоте обращений. Лучшая на неравномерной (Zipf-подобной) нагрузке; медленно адаптируется.
- **Random** — случайная жертва. $O(1)$, без метаданных; опорная точка «ценности истории».
- **NoEviction** — после заполнения отвергает вставки. Хороша, если рабочее множество помещается в кэш.

LRU и LFU кодируют два разных признака — давность и частоту.

Профили нагрузки

- **Uniform** — равновероятный доступ; нет закономерности.
- **Zipf** — сильно неравномерная: $\sim$20 % ключей дают $\sim$80 % обращений ($\alpha = 1{,}2$). «Горячие» ключи.
- **Sequential Loop** — циклический проход $0, 1, \ldots, N{-}1, 0, \ldots$; при $N > C$ — плохой случай для LRU.
- **Read-Heavy** / **Write-Heavy** — 90 % / 10 % чтений.
- **Burst** — 80 % обращений к малому горячему подмножеству.

Метрики: hit ratio, throughput, p_{50}/p_{99} латентности Get, число вытеснений, оценка памяти. Каждая конфигурация — среднее по 10 запускам.

Архитектура и потокобезопасность

Хранение: `unordered_map` (доступ $O(1)$) + отделённый индекс политики (`OnInsert/OnAccess/OnErase/Victim`); TTL — упорядоченный индекс.

- LRU — список + хеш-индекс, $O(1)$;
- LFU — бакеты частот, $O(\log U)$;
- Random — массив + индекс позиций, $O(1)$.

Синхронизация: единый `std::mutex` на все публичные методы.

- корректность видна из устройства (одна критическая секция на операцию);
- подтверждена 36 тестами, включая 5 многопоточных стресс-тестов.

Плата: единый замок сериализует операции — узкое место по масштабированию (см. результаты).

Более тонкие блокировки

Проблема. Единый мьютекс $\Rightarrow$ операции кэша не идут параллельно; с ростом числа потоков выигрыш быстро упирается в конкуренцию за замок.

Решение. Разбить кэш на независимые части (шарды) по ключу, каждую под своим замком $\Rightarrow$ параллелизм $\sim$числу шардов.

Компромисс. Общие данные (учёт памяти, индекс политики, TTL) остаются разделяемыми и требуют отдельной синхронизации. В работе выбран простой и заведомо корректный вариант; шардирование — направление развития.

Бенчмарк: cache-aside с задержкой хранилища

- Шаблон **cache-aside**: Get → кэш; при промахе → хранилище → запись в кэш. Set обновляет хранилище и кэш.
- Задержка доступа к хранилищу 2 µs на промах и на запись — моделирует реальный backing store. Промах дорог, попадание дёшево ⇒ throughput и latency зависят от hit ratio. Применяется без блокировок (промахи перекрываются).
- **Единый пул** из T потоков: каждый берёт следующую операцию из общей очереди ⇒ соотношение чтений/записей держится весь прогон.
- **Перестановка ключей**: горячий набор не совпадает с порядком прогрева ⇒ NoEviction не получает незаслуженного преимущества.

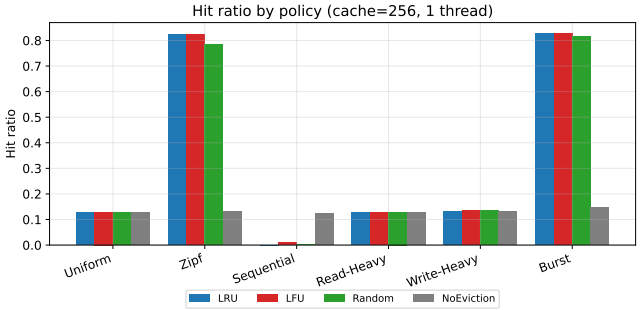
Параметры и гипотезы

План: 4 политики $\times$ 6 профилей $\times$ 6 размеров кэша ($64 \dots 2048$, $N = 2000$) $\times$ потоки $\{1, 2, 4, 8\}$, по 10 запусков — 576 конфигураций.

Гипотезы:

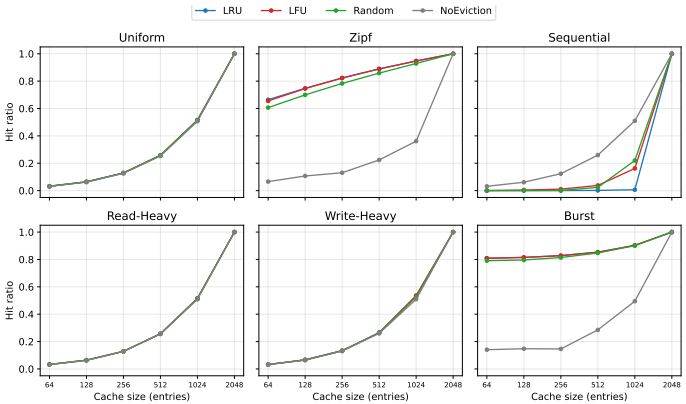
- H1 (Uniform): все политики $\approx C/N$, различий нет.
- H2 (Zipf): выигрывает LFU; порядок LFU $\geq$ LRU $>$ Random $\gg$ NoEviction.
- H3 (Sequential): LRU $\to 0$; выигрывает NoEviction (заморозка).
- H4/H5 (Read/Write-Heavy): $\approx C/N$, как H1.
- H6 (Burst): высокий hit ratio, выигрывают LFU/LRU.

Hit ratio: три режима



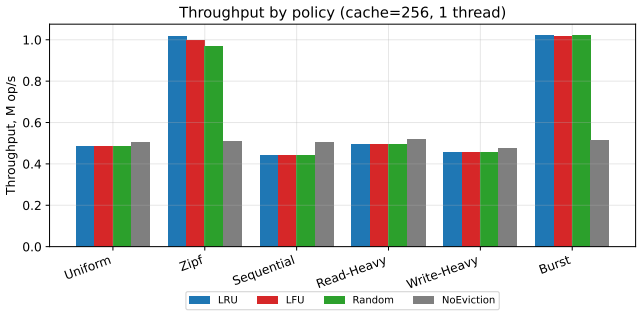
- Uniform / Read / Write-Heavy: все $\approx C/N$ $(0{,}13)$.
- Zipf / Burst: LRU/LFU/Random $0{,}8$–$0{,}9$, NoEviction — провал.
- Sequential: LRU/LFU/Random ≈ 0, NoEviction лидирует.

Hit ratio vs размер кэша



- Uniform: линейно $\approx C/N$.
- Zipf: вогнуто, насыщение (ядро ловится малым кэшем).
- Sequential: у LRU «плато у нуля» до $C \approx N$.
- При $C \geq N$ все сходятся к $1{,}0$.

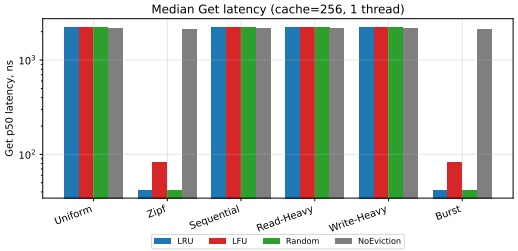
Throughput следует за hit ratio

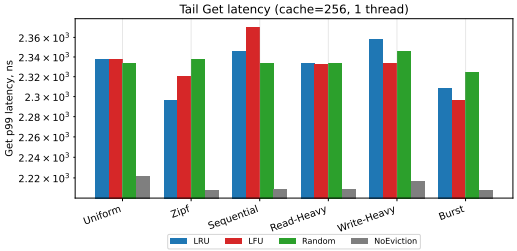


Каждый промах стоит 2 µs ⇒

- Zipf/Burst (высокий hit) вдвое быстрее uniform/seq;
- Random не обгоняет LRU/LFU;
- NoEviction — самый медленный на Zipf/Burst: низкий hit ratio не окупается дешевизной вытеснения.

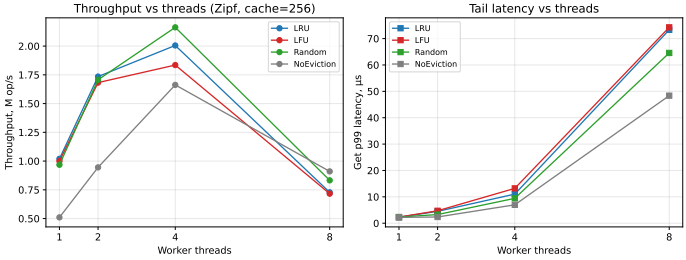
Латентность Get: p_{50} и p_{99}





Попадание — десятки нс; промах — $\sim$2 µs (обращение к хранилищу). На Zipf/Burst медиана низкая (попадание), у NoEviction — высокая (медианный Get у него промах).

Масштабирование по потокам (Zipf, $C = 256$)



- Ожидания промахов перекрываются $\Rightarrow$ рост до 2–4 потоков.
- Дальше спад: путь попадания сериализуется единым мьютексом.
- p_{99} растёт на порядок.

Прямой довод в пользу гранулярных блокировок.

Сводная таблица (выдержка, $C = 256$, 1 поток)

Профиль	Лучшая по hit ratio	hit	Throughput	Замечание
Uniform	все равны	$0{,}13$	$\sim 0{,}5$	политика не важна
Zipf	**LFU/LRU**	$0{,}84$	$\sim 1{,}0$	NoEviction провал
Sequential Loop	**NoEviction**	$0{,}12$	$\sim 0{,}5$	LRU $\to 0$
Read-Heavy	все равны	$0{,}13$	$\sim 0{,}5$	как Uniform
Write-Heavy	все равны	$0{,}13$	$\sim 0{,}45$	запись всегда в хранилище
Burst	**LFU/LRU**	$0{,}83$	$\sim 1{,}0$	NoEviction провал

Throughput в M оп/с. Полные таблицы — в отчёте.

Гипотезы vs результаты

- H1 (Uniform) — **подтверждена**: все $\approx C/N$.
- H2 (Zipf) — **подтверждена**: LFU $>$ LRU $>$ Random $\gg$ NoEviction.
- H3 (Sequential) — **подтверждена**: LRU $\to 0$, NoEviction лучший.
- H4/H5 (Read/Write-Heavy) — **подтверждены**: $\approx C/N$; Read-Heavy быстрее Write-Heavy.
- H6 (Burst) — **подтверждена**: высокий hit ratio у LFU/LRU/Random.

Все гипотезы согласуются с теорией и литературой.

Рекомендации по выбору политики

- **Скошенная популярность (Zipf, Burst)** — LFU (при смене горячих ключей — адаптивные политики).
- **Циклическое сканирование (**$N > C$**)** — NoEviction или scan-устойчивая стратегия; чистый LRU противопоказан.
- **Равномерная нагрузка (Uniform, Read/Write-Heavy)** — политика не важна; ключ к hit ratio — размер кэша, а не выбор политики.
- **Высокая конкуренция потоков** — единый мьютекс ограничивает масштабирование; нужны гранулярные (шардированные) блокировки.

Спасибо за внимание!

Вопросы?

https://github.com/vlklk/team19-key-value-cache